

Achievement System

Slime Rush 기반 업적 조건 판정 / 저장 / UI 피드백 시스템 | 김철규

Steam 출시 게임의 업적 시스템 경험을 기반으로 작성한 공개용 샘플입니다. 조건 판정, 선행 조건, 저장소 분리, 로컬/클라우드 저장, UI 이벤트까지 하나의 흐름으로 정리했습니다.

Achievement ``Repository`` Save Data ``Cloud Sync`` UI Feedback

Sheet Data

Create

Report

Complete

Save/UI

샘플 목표

- 업적 정의 데이터와 저장 데이터를 분리
- 조건 타입별 진행도 누적과 완료 판정
- 선행 업적 완료 후 활성화
- Repository로 저장소 교체 가능
- 업적 완료 이벤트로 UI 자동 갱신

원 프로젝트 연결

- Slime Rush 업적 시스템
- 로컬/클라우드 저장 대응
- UGUI 팝업 피드백
- Unity Localization 연동
- Steam 출시 프로젝트 시스템 개발 경험

역할 업적 시스템 설계 및 구현

기술 C#, Unity, Repository Pattern, Observer Pattern, JSON Save

범위 데이터 모델, 조건 판정, 상태 전환, 저장/로드, 팝업 UI 이벤트

업적 정의 데이터와 저장 데이터 분리

```
public enum AchievementState { Inactive, Active, Completed }
public enum AchievementConditionType { None, StageClear, GetMagic, RuneMaxLevel, KillMonster }

public sealed class AchievementSheetData
{
    public string Id { get; init; }
    public string NameKey { get; init; }
    public string DescriptionKey { get; init; }
    public AchievementConditionType ConditionType { get; init; }
    public string ConditionValue { get; init; }
    public int RequiredCount { get; init; }
    public string[] Prerequisites { get; init; } = Array.Empty<string>();
}

public sealed class AchievementSaveData
{
    public string Id { get; init; }
    public int CurrentCount { get; set; }
    public bool IsCompleted { get; set; }
}
```

정의 데이터

밸런스/기획 데이터입니다. 이름, 설명, 조건 타입, 목표 수치, 선행 조건을 포함합니다.

저장 데이터

플레이어별 진행 상태입니다. 현재 카운트와 완료 여부만 저장해 데이터 크기를 줄입니다.

개별 업적 객체와 상태 전환

```
public sealed class Achievement
{
    public string Id { get; }
    public AchievementConditionType ConditionType { get; }
    public string ConditionValue { get; }
    public int RequiredCount { get; }
    public IReadOnlyList<string> Prerequisites { get; }
    public int CurrentCount { get; private set; }
    public AchievementState State { get; private set; }
    public event Action<Achievement> Completed;

    public Achievement(AchievementSheetData sheet, AchievementSaveData save)
    {
        Id = sheet.Id;
        ConditionType = sheet.ConditionType;
        ConditionValue = sheet.ConditionValue;
        RequiredCount = sheet.RequiredCount;
        Prerequisites = sheet.Prerequisites;
        CurrentCount = save?.CurrentCount ?? 0;
        State = save?.IsCompleted == true ? AchievementState.Completed : AchievementState.Inactive;
    }

    public void Activate()
    {
        if (State == AchievementState.Inactive)
            State = AchievementState.Active;
    }

    public bool Report(AchievementReport report)
    {
        if (State != AchievementState.Active || !Matches(report)) return false;
        CurrentCount = Math.Min(RequiredCount, CurrentCount + report.Count);
        if (CurrentCount < RequiredCount) return false;
        State = AchievementState.Completed;
        Completed?.Invoke(this);
        return true;
    }
}
```

리뷰 포인트

업적 객체가 자기 상태 전환을 책임집니다. 외부 시스템은 이벤트를 보고 UI/저장을 처리하면 됩니다.

게임 이벤트를 업적 리포트로 정규화

```
public readonly struct AchievementReport
{
    public AchievementConditionType Type { get; init; }
    public string Value { get; init; }
    public int Count { get; init; }

    public static AchievementReport StageClear(string stageId)
        => new() { Type = AchievementConditionType.StageClear, Value = stageId, Count = 1 };

    public static AchievementReport GetMagic(string magicId)
        => new() { Type = AchievementConditionType.GetMagic, Value = magicId, Count = 1 };

    public static AchievementReport KillMonster(string monsterId, int count)
        => new() { Type = AchievementConditionType.KillMonster, Value = monsterId, Count = count };
}

private bool Matches(AchievementReport report)
{
    if (ConditionType != report.Type) return false;
    if (ConditionValue == "*" || string.IsNullOrEmpty(ConditionValue)) return true;
    return ConditionValue == report.Value;
}
```

문제

게임 이벤트 종류가 많아질수록 업적 시스템이 각 게임 시스템을 직접 알게 되기 쉽습니다.

해결

스테이지 클리어, 마법 획득, 몬스터 처치 등을 공통 리포트 구조로 정규화합니다.

업적 시스템의 중앙 관리 흐름

```
public sealed class AchievementSystem
{
    private readonly IAchievementRepository _repository;
    private readonly List<Achievement> _achievements = new();
    public event Action<Achievement> AchievementCompleted;

    public AchievementSystem(IAchievementRepository repository)
    {
        _repository = repository;
    }

    public async Task InitializeAsync()
    {
        var sheets = await _repository.LoadSheetDataAsync();
        var saves = await _repository.LoadSaveDataAsync();
        _achievements.AddRange(AchievementCreator.Create(sheets, saves));

        foreach (var achievement in _achievements)
            achievement.Completed += OnCompleted;

        UpdateActivation();
    }

    public void Report(AchievementReport report)
    {
        foreach (var achievement in _achievements)
            achievement.Report(report);
    }

    private void OnCompleted(Achievement achievement)
    {
        UpdateActivation();
        _repository.SaveAsync(CreateSaveData());
        AchievementCompleted?.Invoke(achievement);
    }
}
```

실무 효과

게임 시스템은 Report만 호출합니다. 저장, 활성화, UI 이벤트는 AchievementSystem 내부에서 일관되게 처리합니다.

선행 조건과 자동 활성화

```
private void UpdateActivation()
{
    var completedIds = _achievements
        .Where(x => x.State == AchievementState.Completed)
        .Select(x => x.Id)
        .ToHashSet();

    foreach (var achievement in _achievements)
    {
        if (achievement.State != AchievementState.Inactive)
            continue;

        var canActivate = achievement.Prerequisites.All(completedIds.Contains);
        if (canActivate)
            achievement.Activate();
    }
}

private IReadOnlyList<AchievementSaveData> CreateSaveData()
{
    return _achievements
        .Select(x => new AchievementSaveData
        {
            Id = x.Id,
            CurrentCount = x.CurrentCount,
            IsCompleted = x.State == AchievementState.Completed
        })
        .ToArray();
}
```

활성화 정책

선행 업적이 모두 완료된 업적만 Active 상태로 전환합니다.

저장 정책

정의 데이터는 저장하지 않고 플레이어 진행 상태만 저장합니다.

저장소 추상화와 로컬/클라우드 분리

```
public interface IAchievementRepository
{
    Task<IReadOnlyList<AchievementSheetData>> LoadSheetDataAsync();
    Task<IReadOnlyList<AchievementSaveData>> LoadSaveDataAsync();
    Task SaveAsync(IReadOnlyList<AchievementSaveData> data);
}

public interface IAchievementDataSource
{
    Task<string> LoadAsync();
    Task SaveAsync(string json);
}

public sealed class AchievementRepository : IAchievementRepository
{
    private readonly IAchievementDataSource _local;
    private readonly IAchievementDataSource _cloud;

    public async Task<IReadOnlyList<AchievementSaveData>> LoadSaveDataAsync()
    {
        var cloudJson = await _cloud.LoadAsync();
        if (!string.IsNullOrEmpty(cloudJson))
            return JsonConvert.DeserializeObject<List<AchievementSaveData>>(cloudJson);

        var localJson = await _local.LoadAsync();
        return string.IsNullOrEmpty(localJson)
            ? Array.Empty<AchievementSaveData>()
            : JsonConvert.DeserializeObject<List<AchievementSaveData>>(localJson);
    }

    public async Task SaveAsync(IReadOnlyList<AchievementSaveData> data)
    {
        var json = JsonConvert.SerializeObject(data);
        await _local.SaveAsync(json);
        await _cloud.SaveAsync(json);
    }
}
```

업적 완료 이벤트와 팝업 UI

```
public sealed class AchievementPopupPresenter : MonoBehaviour
{
    [SerializeField] private AchievementPopupItem itemPrefab;
    [SerializeField] private Transform contentRoot;
    [SerializeField] private LocalizedStringTable stringTable;

    private AchievementSystem _achievementSystem;

    public void Initialize(AchievementSystem achievementSystem)
    {
        _achievementSystem = achievementSystem;
        _achievementSystem.AchievementCompleted += ShowCompleted;
    }

    private void ShowCompleted(Achievement achievement)
    {
        var item = Instantiate(itemPrefab, contentRoot);
        item.Bind(new AchievementPopupViewModel
        {
            Name = stringTable.Get(achievement.NameKey),
            Description = stringTable.Get(achievement.DescriptionKey),
            State = achievement.State
        });
        item.PlayOpenAnimation();
    }

    private void OnDestroy()
    {
        if (_achievementSystem != null)
            _achievementSystem.AchievementCompleted -= ShowCompleted;
    }
}
```

리뷰 포인트

도메인 시스템은 UI를 직접 알지 않고 이벤트만 발행합니다. Presenter가 이벤트를 구독해 팝업을 생성합니다.

테스트 가능한 항목

```
// Example test cases
// 1. StageClear report increments only matching stage achievement.
// 2. Completed achievement does not increment again.
// 3. Prerequisite achievement activates next achievement.
// 4. Cloud save is preferred over local save when both exist.
// 5. AchievementCompleted event fires once.

var achievement = new Achievement(sheet, save);
achievement.Activate();

var completed = achievement.Report(AchievementReport.StageClear("stage_01"));

Assert.True(completed);
Assert.Equal(AchievementState.Completed, achievement.State);
Assert.Equal(sheet.RequiredCount, achievement.CurrentCount);
```

테스트 포인트

- 조건 타입 매칭
- 진행도 누적
- 완료 이벤트
- 선행 조건 활성화
- 저장소 우선순위

면접 포인트

- 보상/퀘스트 시스템으로 확장 가능
- UI와 도메인 분리
- 클라우드 저장 충돌 정책 설명 가능
- Localization 연동 구조 설명 가능

코드 리뷰에서 보여주고 싶은 점

강점

- 정의 데이터와 저장 데이터 분리
- 게임 이벤트를 Report로 정규화
- Repository로 저장소 의존성 분리
- 이벤트 기반 UI 피드백
- 선행 조건 기반 상태 전환

게임 시스템 확장

- 업적
- 퀘스트
- 미션
- 일일 과제
- 배틀패스 진행도

포트폴리오 연결 문구

이 샘플은 Slime Rush 업적 시스템 경험을 바탕으로 구성했습니다. 실제 프로젝트에서는 업적 조건 판정, 진행 상태 저장, 로컬/클라우드 저장, UI 피드백을 담당했습니다.

연락처

Portfolio: <https://cosmicgiantkoala.github.io> Email: cosmicgiantkoala@gmail.com